AI Agent Replication Guide: Project Workspace, Import/Export, Icons, and Layout

Date: 2026-10-20
Audience: AI coding agent implementing an OPTransformer-like UI shell

Mission

Replicate the same design and interaction model for:
1. Project creation and reset workflow
2. Project import/export workflow (single + bulk)
3. Icon system used across toolbar and activity rail
4. Full application layout (IDE-like shell)

This guide is implementation-oriented and includes code templates.

---

1) Authoritative behavior model

Implement the behavior exactly as follows (based on active app code):

- Project state is centrally managed in the root app shell.
- Save persists to backend first, local storage second.
- Persisted project state stores lightweight schema references (not full schema bodies).
- Import supports multiple JSON envelope formats and merges safely.
- Import merge resolves name conflicts with suffixes.
- Export supports all projects and single project payloads.

```
  mappings: MappingModel[];
  activeMappingId: string;
};

type PersistedProject = {
  id: string;
  name: string;
  updatedAt: string;
  state: ProjectState;
};

type SchemaRef = {
  id: string | null;
  messageType: string | null;
  version: string | null;
  namespace: string | null;
  sourceType: string; // default XSD
};

type LoadedMessage = {
  id: string;
  messageType: string;
  version: string;
  namespace?: string;
  sourceType?: string;
  schema?: any;       // hydrated runtime body
```

- Persisted project state stores lightweight schema references (not full schema bodies).
- Import supports multiple JSON envelope formats and merges safely.
- Import merge resolves name conflicts with suffixes.
- Export supports all projects and single project payloads.
- Layout is a dark IDE shell with top toolbar + left icon rail + side panel + content pane + overlays.

---

2) Required project state contract

Use this canonical state shape in the root container:

```ts
type ProjectState = {
  messageType: string;
  version: string;
  uploadMessageType: string;
  uploadVersion: string;
  uploadNamespace: string;
  search: string;
  schema?: any;                // full body in-memory only
  schemaRef?: SchemaRef | null; // persisted
  loadedMessages: LoadedMessage[]; // persisted in slim form
  activeMessageId: string;
  mappings: MappingModel[];
  activeMappingId: string;
```

```
  sourceType?: string;
  schema?: any;       // hydrated runtime body
  schemaRef?: SchemaRef | null;
};
```

Key principle:
- Persist only `schemaRef` + slim message metadata.
- Re-fetch schema body on project load.

---

3) Project creation workflow

3.1 New project (hard reset)

On `New Project` action:
- Clear project identity (`projectName`, `selectedProjectId`)
- Clear schema/search/errors/upload fields
- Reset message defaults (`messageType='pacs.008'`, `version='001.08'`)
- Clear loaded messages, mappings, active IDs
- Set status message: `Started a new project`

Template logic:

```javascript
function newProject() {
```

````
```javascript
function newProject() {
  setProjectName('');
  setSelectedProjectId('');
  setSchema(null);
  setSearch('');
  setError(null);
  setMessageType('pacs.008');
  setVersion('001.08');
  setUploadFile(null);
  setUploadMessageType('uploaded-xsd');
  setUploadVersion('');
  setUploadNamespace('');
  setLoadedMessages([]);
  setActiveMessageId('');
  setMappings([]);
  setActiveMappingId('');
  setProjectStatus('Started a new project');
}
```

## 3.2 Save project

Rules:
- Require non-empty project name.
- Build full in-memory state.
````

- Convert to persistable slim state.
- Upsert project by selected ID, else by same name, else create new ID.
- Save via `persistProjects(nextProjects)`.

ID strategy:
- New IDs: `project-${Date.now()}`

Persist message:
- `Saved project "<name>" (schema bodies are re-fetched on load)`

---

4) Project import/export workflow

4.1 Export all projects

Output payload:

```json
{
  "kind": "optransformer-projects",
  "version": 1,
  "exportedAt": "2026-10-20T10:00:00.000Z",
  "projects": []
}
```

```

Behavior:
- Download file name: `optransformer-projects-<timestamp>.json`
- Also copy JSON to clipboard (best-effort)

## 4.2 Export single project

Output payload:

```json
{
  "kind": "optransformer-project",
  "version": 1,
  "exportedAt": "2026-10-20T10:00:00.000Z",
  "project": {
    "id": "project-123",
    "name": "Payments Mapping",
    "updatedAt": "...",
    "state": {}
  }
}
```

Behavior:
- File name: `optransformer-project-<safeProjectName>.json`
```

```
171
172  Behavior:
173  - File name: `optransformer-project-<safeProjectName>.json`
174
175  ## 4.3 Accepted import formats
176
177  Parser must accept:
178  1. Raw array: `[ {project...}, ... ]`
179  2. Wrapped bulk: `{ projects: [...] }`
180  3. Wrapped single: `{ kind: 'optransformer-project', project: {...} }`
181  4. Bare single project object: `{ id, state, ... }`
182
183  Reject any other format with clear error.
184
185  ## 4.4 Merge algorithm
186
187  Given `incoming[]` and current `projects[]`:
188  - Index current projects by `id`.
189  - Track used names (case-insensitive).
190  - For each incoming project:
191    - Ensure ID (generate random fallback if missing).
192    - Resolve duplicate name for new IDs by suffixing ` (2)`, ` (3)`, ...
193    - Upsert into map.
194  - Sort by `updatedAt` descending.
195  - Persist merged list.
196  - Status: `Imported X new, replaced Y project(s)`.
```

```
    if (byId.has(id)) replaced += 1;
    else added += 1;

    byId.set(id, entry);
    usedNames.add(name.toLowerCase());
  });

  const merged = Array.from(byId.values()).sort((a, b) =>
    String(b.updatedAt || '').localeCompare(String(a.updatedAt || ''))
  );

  persistProjects(merged);
  return { added, replaced, merged };
}
```

---

5) Persistence architecture

5.1 Primary + fallback storage

Use two-layer persistence:

1. Primary backend endpoint:

Template implementation:

```javascript
function mergeImportedProjects(incoming, currentProjects, persistProjects) {
  const byId = new Map(currentProjects.map((p) => [p.id, p]));
  const usedNames = new Set(currentProjects.map((p) => (p.name || '').toLowerCase()));
  let added = 0;
  let replaced = 0;

  incoming.forEach((p) => {
    if (!p || typeof p !== 'object') return;
    const id = p.id || `project-${Date.now()}-${Math.random().toString(36).slice(2, 8)}`;
    let name = p.name || 'Imported project';

    if (!byId.has(id) && usedNames.has(name.toLowerCase())) {
      let n = 2;
      while (usedNames.has(`${name} (${n})`.toLowerCase())) n += 1;
      name = `${name} (${n})`;
    }

    const entry = {
      id,
      name,
      updatedAt: p.updatedAt || new Date().toISOString(),
      state: p.state || {},
    };
```

```
## 6) Hydration on project load

When user opens project:

1. Set base UI state (`projectName`, `messageType`, etc.)
2. Hydrate `loadedMessages` by resolving `schemaRef`
3. Resolve active schema by:
   - inline schema if old project
   - `state.schemaRef`
   - active message schema fallback
4. Restore mappings and active mapping
5. Switch view to schema tab
6. Emit status and mention any missing schema body re-fetch failures

Schema fetch strategy:
- Try by ID endpoint first
- Fallback to messageType/version endpoint

---

## 7) Icon system and semantic map

Use the following emoji icon registry to replicate the same feel.

## 7.1 Top toolbar icons
```

```
247  Use two-layer persistence:
248
249  1. Primary backend endpoint:
250  - `POST /schema/projects/save-all`
251  - body: serialized project array
252
253  2. Secondary cache:
254  - `localStorage['xsd-ui-projects-v1']`
255
256  ## 5.2 Safe serialization
257
258  Use circular-safe JSON serialization:
259  - skip circular objects
260  - skip functions
261  - skip DOM nodes/window objects
262
263  ## 5.3 Startup restore flow
264
265  On app mount:
266  1. Try `GET /schema/projects/load-all`
267  2. If empty/fail, fallback to localStorage JSON parse
268
269  ---
270
271  ## 6) Hydration on project load
272
273  When user opens project:
```

optransformer > docs > M↓ AI Agent Project Layout Replication Guide.md

```
## 7.1 Top toolbar icons

- New project: `📄`
- Save project: `💾`
- Export all projects: `📤`
- Import projects from file: `📥`
- Paste JSON import: `📋`
- Load schema: `▶`
- Upload and parse schema: `⬆`
- Refresh: `🔄`
- SWIFT MT: `✉`
- ACH/NACHA: `🏦`
- Mapping intelligence: `🧠`
- Import field structure: `📥`
- MT-to-MX preview: `⚙ MX`
- New mapping: `🔀`
- New message class: `📦`
- Project indicator: `📁 <projectName>`

## 7.2 Left activity rail icons

- Project: `📁`
- Load Schema: `📄`
- Upload Schema: `⬆`
- Mappings: `🔀`
- SWIFT MT: `✉`
```

optransformer > docs > M↓ AI_Agent_Project_Layout_Replication_Guide.md

```
313
314  ## 7.2 Left activity rail icons
315
316  - Project: `📁`
317  - Load Schema: `📄`
318  - Upload Schema: `⬆`
319  - Mappings: `🔀`
320  - SWIFT MT: `📨`
321  - ACH/NACHA: `🏦`
322  - Schema Workbench: `🧰`
323  - Product Owner Portal: `⚡`
324  - Flow Builder: `🔧`
325  - Flow Orchestrator: `🧩`
326  - Transformation Package Studio: `📦`
327  - ICD Workbench: `📋`
328  - Mapping Intelligence: `🧠`
329  - ICD Visualizer: `📊`
330  - Cartridge Intelligence: `🗂️`
331  - Mapping Document Builder: `🧭`
332  - Message Class Studio: `🧱`
333  - Sample Mapping: `🔬`
334  - CAR Mapping Studio: `🚗`
335  - AI Assistant: `🤖` (feature/session gated)
336
337  ---
338
339  ## 8) Overall layout blueprint (pixel-level intent)
```

optransformer > docs > ML AI Agent Project Layout Proposal

- CAR Mapping Studio: `🚗`
- AI Assistant: `🤖` (feature/session gated)

---

8) Overall layout blueprint (pixel-level intent)

Root shell:
- Full viewport (`height: 100vh`)
- Dark theme background `#1e1f22`
- Monospace UI style (`JetBrains Mono`, `Fira Code`)

Structure:

1. Top toolbar
- Height: `32px`
- Background: `#2b2d30`
- Bottom border: `#1e1f22`

2. Main row
- Left activity strip width: `36px`
- Side panel (toggle by tool)
- Main content flex area

3. Secondary action bar in content
- Height: `28px`
- Background: `#3c3f41`

3. Secondary action bar in content
- Height: `28px`
- Background: `#3c3f41`
- Contains title, search, role badge, admin button, logout, contextual actions

4. Content area
- Padding: `12px`
- Scrollable

5. Overlay/modal layer (mounted at root)
- loaders/workbench/portal/flow/mapping/assistant/admin/confirm/undo

---

9) Reference implementation code (attach to AI agent)

9.1 Toolbar component skeleton

```jsx
function Toolbar({
  onNewProject,
  onSaveProject,
  onExportProjects,
  onImportProjectsFile,
  onImportProjectsPrompt,
```

```
  projectName,
  projectStatus,
}) {
  // keep button style compact (26x24)
  // icons as specified in section 7.1
  // hidden file input for project import
}
```

9.2 Activity rail registry

```javascript
const activityTools = [
  { id: 'project', icon: '📁', title: 'Project' },
  { id: 'schema', icon: '📄', title: 'Load Schema' },
  { id: 'upload', icon: '⬆', title: 'Upload Schema' },
  { id: 'mapping', icon: '🔀', title: 'Mappings' },
  { id: 'swiftmt', icon: '📨', title: 'SWIFT MT' },
  { id: 'ach', icon: '🏦', title: 'ACH / NACHA' },
  { id: 'workbench', icon: '🧰', title: 'Schema Workbench' },
  { id: 'po-portal', icon: '⚡', title: 'Product Owner Portal' },
  { id: 'flow-builder', icon: '🔧', title: 'Flow Builder' },
  { id: 'flow-orchestrator', icon: '🎛', title: 'Flow Orchestrator' },
  { id: 'mt-package', icon: '📦', title: 'Transformation Package Studio' },
  { id: 'icd-workbench', icon: '📋', title: 'ICD Workbench' },
  { id: 'mapping-intelligence', icon: '🧠', title: 'Mapping Intelligence' },
```

```
function Toolbar({
  onNewProject,
  onSaveProject,
  onExportProjects,
  onImportProjectsFile,
  onImportProjectsPrompt,
  importProjectsInputRef,
  onLoadSchema,
  onUploadAndParseSchema,
  onRefresh,
  onNewMapping,
  onLoadSwiftMt,
  onLoadAch,
  onOpenMappingIntelligence,
  onImportFieldStructure,
  onOpenMtToMx,
  messageType,
  setMessageType,
  version,
  setVersion,
  versions,
  versionsLoading,
  loading,
  uploadFile,
  projectName,
  projectStatus,
) {
```

mer > docs > M↓ AI_Agent_Project_Layout_Replication_Guide.md

```
}

export function buildSingleProjectPayload(project) {
  return {
    kind: 'optransformer-project',
    version: 1,
    exportedAt: new Date().toISOString(),
    project,
  };
}
```

---

10) AI agent implementation checklist (must pass)

- [ ] New project resets all authoring state and statuses correctly.
- [ ] Save project persists slim schema refs only.
- [ ] Export all and export single generate exact envelope format.
- [ ] Import handles all supported payload formats.
- [ ] Merge import resolves name collisions and upserts by ID.
- [ ] Restore on startup tries backend, then localStorage fallback.
- [ ] Activity rail icons match registry and behavior.
- [ ] Layout dimensions/colors align with shell blueprint.
- [ ] Modal/overlay mounting pattern is root-level and non-breaking.

```
{ id: 'icd-visualizer', icon: '📊', title: 'ICD Visualizer' },
{ id: 'cartridge-intelligence', icon: '🟨', title: 'Cartridge Intelligence' },
{ id: 'mapping-doc-builder', icon: '🧭', title: 'Mapping Document Builder' },
{ id: 'message-class', icon: '🔷', title: 'Message Class Studio' },
{ id: 'sample-mapping', icon: '🔬', title: 'Generate Mapping from Samples' },
{ id: 'car-mapping-studio', icon: '🟦', title: 'CAR to Mapping Studio' },
{ id: 'ai-assistant', icon: '🤖', title: 'AI Assistant' },
];
```

9.3 Persist + import/export utility skeleton

```javascript
export function parseImportedJson(raw) {
  const data = JSON.parse(raw);
  if (Array.isArray(data)) return data;
  if (data && Array.isArray(data.projects)) return data.projects;
  if (data && data.kind === 'optransformer-project' && data.project) return [data.project];
  if (data && data.id && data.state) return [data];
  throw new Error('Unrecognized format (expected project, array, or {projects: []})');
}

export function buildAllProjectsPayload(projects) {
  return {
    kind: 'optransformer-projects',
    version: 1,
    exportedAt: new Date().toISOString(),
```

13) Handover prompt for another AI agent

```text
Implement an IDE-style React shell matching OPTransformer behavior for project lifecycle and layout.

Hard requirements:
1) Project lifecycle
- new project reset
- save with slim schema refs
- load with schema hydration

2) Import/export
- support these formats: array, {projects}, {kind:optransformer-project,project}, bare single object
- export bulk and single payload envelopes exactly
- merge by id; rename duplicate names with numeric suffixes

3) UI and icons
- use the exact toolbar and activity icon registry from this document
- keep dimensions/spacing/colors aligned to dark IDE shell

4) Persistence
- POST /schema/projects/save-all primary
- localStorage fallback key xsd-ui-projects-v1
- startup load from /schema/projects/load-all then localStorage

5) Acceptance
- pass all scenarios in section 11
```